

Zero Trust-Enabled SDN Architecture for Secure and Intelligent Load Balancing in Edge Networks

Complete panel briefing — what it is, what it proves, where it applies, and what to say

In one paragraph. Every load balancer in production today — from a cloud health-check pool to a Kubernetes scheduler to a 5G edge orchestrator — decides where to send work based on what the servers *say* about themselves. That assumption breaks the moment one of those servers is compromised, rented, misconfigured, or simply selfish. We built an SDN control plane that instead decides from what it **observes**: it continuously scores every edge server on measured behaviour, and when a server misbehaves it revokes that server in the data plane — as real OpenFlow rules, in under a second — while an append-only cryptographic ledger records why. We then ran the same network, same workload and same six attacks *with and without* that mechanism, and measured the difference.

8 / 8ATTACKS DETECTED AND
CORRECTLY FAMILIED**8.2–11.3 s**ONSET → ISOLATION, ALL
FOUR SERVER ATTACKS**0 → 5**HONEST DEVICES STARVED:
OURS VS. CONTROL ARM**4 / 4**NON-FUNCTIONAL
REQUIREMENTS PASS

System under review. An os-ken (Ryu-fork) OpenFlow 1.3 controller application, its REST/SSE northbound API and live dashboard. Mininet, the node agents and the IoT clients are the *test harness*, not the product.

Scale of the measured runs. 1 core + 8 edge OpenVSwitch switches, 8 edge servers, 40 IoT devices, 6 armed attack classes, ~300 s per run, on a 4-core WSL2 box.

Evidence base. 13 live runs; the headline pair is a controlled baseline-vs-treatment comparison recorded 2026-09-05. 600-run seeded statistical harness. 1 004 automated tests (993 pass, 11 skipped, 0 fail).

Every number in this document is traceable to `data/comparison/comparison.txt`, `data/nfr_report.txt`, `data/raft_timeline.jsonl`, `docs/EVALUATION.md` or `pytest -q`. Where a figure is an estimate, a simulation or an upper bound, it says so.

§ Contents

- | | | | |
|----------|--|-----------|---|
| 1 | The problem, in plain language
Why self-reported telemetry is a security hole | 9 | Where this is used — real-world application domains |
| 2 | What we built
The system in one page | 10 | Systems this maps onto
Kubernetes, service mesh, MEC, SD-WAN |
| 3 | Architecture and modules
Five layers, one process | 11 | The business and operations case
Who benefits, and how it is bought |
| 4 | How one task flows end to end
Admission → routing → trust → ledger | 12 | What is genuinely novel here
Three findings that came from building it |
| 5 | The trust model and the anomaly gate
Including our strongest finding | 13 | Engineering decisions we would defend |
| 6 | The six attacks and how each is caught | 14 | The eleven-run story
What broke, and what each failure taught |
| 7 | Results: the controlled comparison
Baseline vs zero-trust, same everything else | 15 | Limitations, stated before anyone asks |
| 8 | Testing, NFRs and consensus evidence | 16 | Anticipated panel questions and answers |
| | | 17 | What to demonstrate live, and the fallbacks |
| | | 18 | Glossary and provenance of every number |

HOW TO USE THIS DOCUMENT

Sections 1, 9, 10 and 11 are the **non-technical** half — the problem, the application domains, the systems it maps onto, and the operational case. Sections 3–8 are the technical evidence. Section 16 is the rehearsal material: read it last and read it twice. If you have ten minutes before you walk in, read the cover page, §5's finding, §7's comparison table, and §15's limitations — that quartet is the whole project.

1 The problem, in plain language

Edge computing pushes computation onto many small, physically exposed, heterogeneously-owned machines: roadside cabinets, factory-floor servers, campus racks, telco base-station sites. Three facts collide there.

1.1 The perimeter model fails at the edge

Classical network security draws a wall and trusts everything inside it. At the edge there is no meaningful inside: a node can be physically opened with a screwdriver, rented to a third party, misconfigured by a subcontractor, or simply be selfish. Classical load balancers — round-robin, least-connections — assume every backend is honest about its state and faithfully performs its work. **Neither assumption survives a single compromised node**, and neither balancer has any mechanism that could ever notice.

1.2 Self-reported telemetry is the industry norm, and it is trivially gameable

This is the sharp end of the problem and it is worth saying slowly to the panel. Almost every real orchestrator schedules on what nodes *claim* about themselves: Kubernetes schedules on kubelet-reported resource metrics, cloud auto-scalers on agent-reported utilisation, MEC orchestrators on advertised host capacity. A node that under-reports its load therefore **attracts** traffic — which is exactly what an attacker wants, whether the goal is to intercept data, degrade the service, or freeload on someone else's SLA.

The attacker does not have to break any cryptography to do this. It just has to lie in a field that everybody believes.

1.3 "Zero Trust" is policy, not implementation, at the network layer

NIST SP 800-207 says *never trust, always verify, continuously* — but it is an architecture document, not a mechanism. Commercial Zero-Trust products overwhelmingly verify **users and client devices at access time**. Verifying the **serving infrastructure itself, continuously, with consequences in the routing path** is much less settled, and that is the gap this project works in.

THE HONEST PROBLEM STATEMENT — MEMORISE THIS SENTENCE

“Can an SDN control plane continuously verify edge servers using **observed** rather than **claimed** behaviour, and enforce the verdict in the data plane fast enough to matter?”

Our answer, at prototype scale: **yes** — with isolation inside a 3-second bound — **and** with the important negative result that score-based trust *alone* is provably insufficient to do it (\$5).

1.4 What we are careful *not* to claim

- We do not solve Zero Trust for edge networks in general. We demonstrate one mechanism — behavioural trust plus SDN enforcement — under six modelled attacks.
- Real adversaries have a larger menu than we cover: collusion between nodes, slow-and-low degradation, adaptive attackers who keep every signal just under threshold, and attacks on the controller itself.
- The evaluation is emulation (Mininet with real Open vSwitch and real OpenFlow), not a field deployment. The correct phrase is “**demonstrated in an emulated network**”, never “proven in production”.

2 What we built

A trust-aware SDN load balancer for OpenFlow 1.3 networks. The controller computes a per-node trust score from observed behaviour, ranks edge servers by an EdgeScore, installs the routing decision as real flow rules, and quarantines misbehaving nodes in the data plane. Every trust event is recorded in a tamper-evident SHA-256 + Merkle ledger.

COMPONENT	WHAT IT DOES	STATE
SDN controller controller/trust_balancer.py	os-ken OpenFlow 1.3 application. Proxy-ARP, per-connection VIP rewrite, a five-band flow-priority table, cookie-scoped rule deletion, OpenFlow meters.	FUNCTIONAL
Dynamic trust model trust_engine/trust_calculator.py	Normalised score $T = \alpha \bar{R} + \beta \bar{B} + \gamma \bar{H} - \delta \bar{A}$ with EMA decay $\lambda=0.85$, plus a separate load-bearing anomaly gate.	FUNCTIONAL
Secure load balancing controller/edge_selector.py	EdgeScore with power-of-two-choices sampling, trust eligibility filter, ϵ -greedy exploration, UCB1 online weight tuning.	FUNCTIONAL
Attack classification controller/attack_classifier.py	Six attacks plus bad-credentials devices, each named rather than merely flagged, by one pure function that the live controller and the offline scorer both call.	FUNCTIONAL
Device authentication security/	PRESENT-80 lightweight challenge-response with single-use nonces (30 s TTL), extended with a provisioned source-IP pin so the fleet key authenticates the <i>device</i> and not merely possession of the key.	FUNCTIONAL
Trust ledger blockchain/	SHA-256 hash chain with a Merkle root per block, batching 10 trust updates. Tamper-evident: an edit is localised to the seam, not merely reported as “chain invalid”. Re-verified independently inside the browser.	FUNCTIONAL
Live dashboard dashboard/index.html	Topology, packet animation driven by real OFPFlowStats counters, flow tables, trust curves, nine metric-vs-time charts, per-cluster panel, tamper-testable ledger ribbon — all over server-sent events.	FUNCTIONAL
Control arm base_model/	A complete no-zero-trust build of the same system — static routing, no quarantine, no admission control, no ledger — so the comparison is an experiment rather than a claim.	INTEGRATED
Evaluation harness evaluation/ (19 tools)	NFR pass/fail, per-interval metrics, confusion matrix with detection latency, service availability, scalability sweep to $N=64$, and a 600-run seeded statistical comparison against four baselines.	INTEGRATED
RAFT consensus blockchain/raft*.py	Leader election, log replication, real TCP transport, a 3-replica live cluster with a recorded leader kill. Safety properties proven by test (R-01-R-18). Not wired into the live controller — see §15.	PARTIAL
DDoS response	Flooding is detected and correctly classified; throttling is not built — it needs a per-client-IP OpenFlow path and today's machinery is per-edge-node. Deferred as real scope rather than half-built.	PARTIAL

OVERALL IMPLEMENTATION STATUS

Approximately **60% of the full project scope** is functional and measured, which is the target for this review. The two partial items are named above and again in §15; neither is hidden, and neither is on the critical path of the claim being made.

2.1 One environment decision to state before you are asked

The original deck specified the **Ryu** SDN framework. This project runs on **os-ken**, the actively-maintained fork of Ryu, with the identical API (`os_ken.*` instead of `ryu.*`). The reason is environmental and hard: the target OS ships only Python 3.14, there is no Ryu package for it, and Ryu cannot run above Python 3.13. There is no `ryu` dependency and no `ryu-manager` anywhere in the repository. Present this as a documented engineering decision, not as Ryu.

3 Architecture and modules

Five layers, one operating-system process. The controller is a single os-ken OpenFlow 1.3 application; the trust engine and the ledger are libraries it calls in-process, not network services.

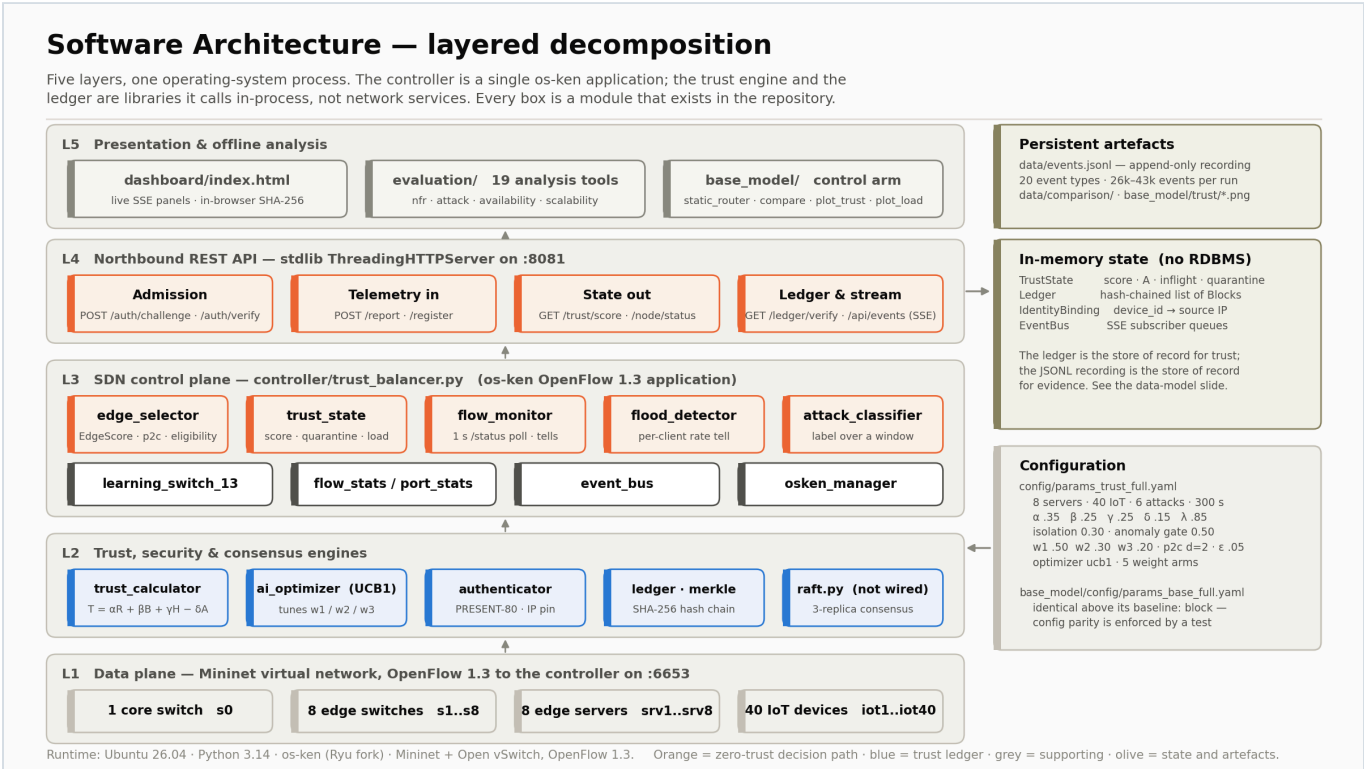


Figure 1. Software architecture — the five layers and the modules in each.

LAYER	MODULES	RESPONSIBILITY
L5 Presentation & analysis	dashboard/, evaluation/ (19 tools), base_model/	Live SSE panels; offline scoring; the no-zero-trust control arm
L4 Northbound API	controller/northbound_api.py	REST + Server-Sent Events on :8081, stdlib ThreadingHTTPServer
L3 SDN control plane	trust_balancer, edge_selector, trust_state, flow_monitor, flood_detector, attack_classifier, event_bus, flow_stats	Routing decisions, detection, enforcement, event publication
L2 Trust, security & consensus	trust_calculator, ai_optimizer, authenticator + present_cipher, ledger.block.merkle, raft*	Trust scoring, weight tuning, admission crypto, the hash-chained ledger
L1 Data plane	simulation/topology, node_agent, iot_client	1 core switch, 8 edge switches, 8 edge servers, 40 IoT devices

THE DESIGN RULE THAT RUNS THROUGH EVERY LAYER

The **request path** writes **evidence** into `TrustState`; the **1-second control loop** reads **decisions** out of it. The two never share a code path. That separation is precisely what makes it possible to run the identical trust code in the control arm with enforcement removed — which is what turns our headline comparison into a controlled experiment rather than two different programs.

3.1 Module interaction

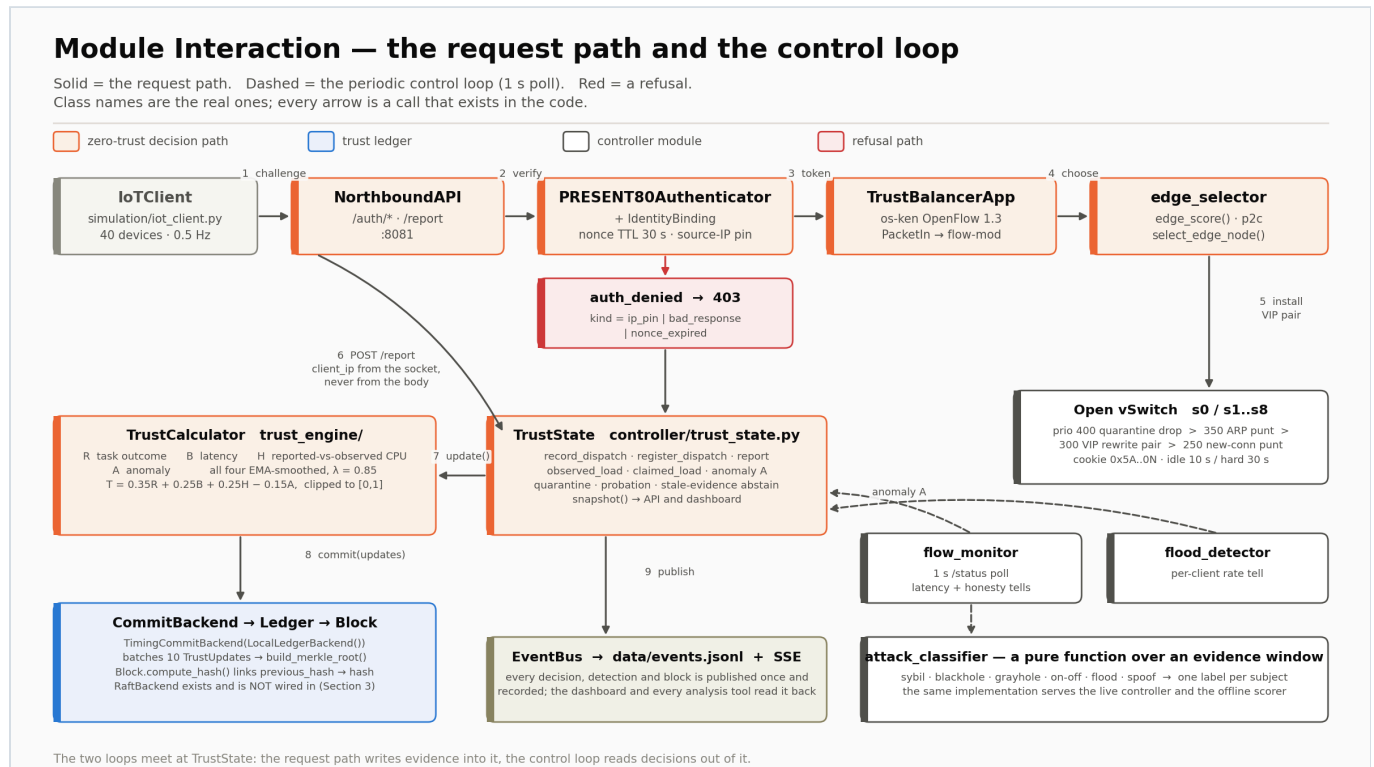


Figure 2. Component and call-flow view: request path, control loop, evidence path.

- **Request path (solid):** `IoTClient` → `NorthboundAPI` → `PRESENT80Authenticator` + `IdentityBinding` → `TrustBalancerApp` → `edge_selector` → OpenFlow flow-mod.
- **Control loop (dashed):** `flow_monitor` (1 s /status poll) and `flood_detector` → anomaly $\bar{A}$ → `TrustState` → `attack_classifier`.
- **Evidence path:** `TrustState` → `TrustCalculator.update()` → `CommitBackend.commit()` → `Ledger` → `Block`; every decision is also published exactly once through `EventBus`.

`attack_classifier` is a **pure function over an evidence window** — no I/O, no locks, no state. The same implementation serves the live controller and the offline scorer, so the live verdict and the marked confusion matrix cannot drift apart.

3.2 The data model — and why there is no relational database

Data Model — entities, keys and relationships

There is no relational database in this system, and the diagram says so rather than inventing one. Trust is stored in an append-only hash-chained ledger; evidence is stored in an append-only JSONL recording. Integrity here is cryptographic, not referential.

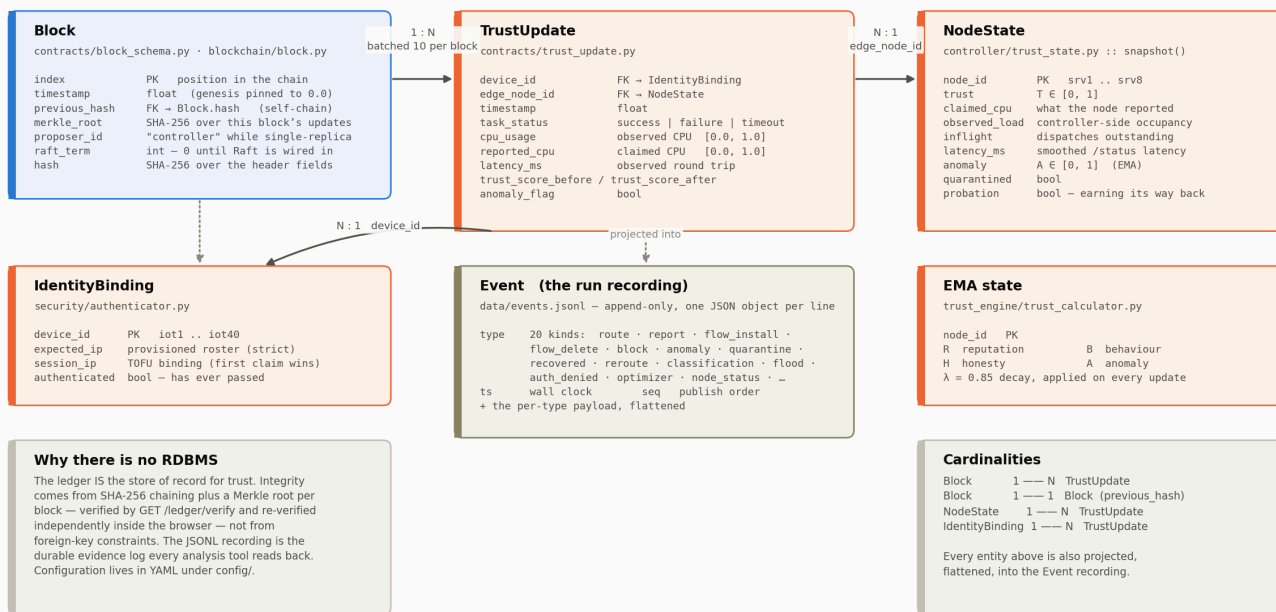


Figure 3. Entity–relationship view of the persisted and in-memory state.

ENTITY	DEFINED IN	KEY FIELDS	INTEGRITY MECHANISM
Block	contracts/block_schema.py	index PK, previous_hash → Block.hash, merkle_root, proposer_id, raft_term	SHA-256 header hash + Merkle root; genesis timestamp pinned to 0.0
TrustUpdate	contracts/trust_update.py	device_id, edge_node_id, task_status, cpu_usage, reported_cpu, latency_ms, anomaly_flag	Merkle leaf; 10 batched per block
NodeState	controller/trust_state.py	node_id PK, trust, observed_load, inflight, anomaly, quarantined, probation	In-memory, invariant-checked: $\text{sum}(\text{inflight}) == \text{len}(\text{dispatches})$
IdentityBinding	security/authenticator.py	device_id PK, expected_ip (roster), session_ip (TOFU)	Source-IP pin, checked on every admission
EMA state	trust_engine/trust_calculator.py	node_id PK, R, B, H, A	$\lambda = 0.85$ exponential decay
Event	data/events.jsonl	type (20 kinds), ts, seq, payload	Append-only; the durable evidence log every analysis tool reads

WHY NO RDBMS — THIS IS A DELIBERATE ANSWER, NOT AN OMISSION

The **ledger is the store of record** for trust, and its integrity is **cryptographic** (hash chain plus Merkle root, verified by `GET /ledger/verify` and re-verified independently in the browser) rather than referential. The JSONL recording is the durable evidence log that every analysis tool reads back offline. Configuration is YAML. Adding a relational database would add a second, weaker source of truth for data whose integrity requirement is tamper-evidence, not joins.

3.3 Northbound API

METHOD	PATH	REQUEST	RESPONSE
POST	<code>/auth/challenge</code>	<code>{device_id}</code>	<code>{nonce}</code> — 64-bit, TTL 30 s, single-use
POST	<code>/auth/verify</code>	<code>{device_id, response}</code>	<code>{token}</code> · 403 + <code>auth_denied{kind}</code>
POST	<code>/report</code>	<code>{device_id, vip_src_port, status, latency_ms}</code>	Task outcome; <code>client_ip</code> read from the socket, never from the body
POST	<code>/register</code>	<code>{node_id, concurrency}</code>	Agent start-up registration
POST	<code>/topology/links</code>	<code>{links:[{a,b,delay_ms,bw_mbps}]}</code>	The link parameters the harness actually built
GET	<code>/trust/score</code>	<code>?node_id=</code>	<code>{node_id: score}</code> or all nodes
GET	<code>/node/status</code>	<code>?node_id=</code>	<code>TrustState.snapshot()</code>
GET	<code>/ledger/verify</code>	—	<code>{valid, chain_length, max_updates_per_block}</code>
GET	<code>/api/events</code>	—	Server-Sent Events stream (one-way, no third-party library)

`auth_denied` kinds are `ip_pin` · `bad_response` · `nonce_expired`. The distinction matters: `ip_pin` versus `bad_response` is the *only* thing that separates a spoofer (correct key, wrong source host) from a device that never held the key at all.

3.4 Deployment

Deployment Architecture — one host, three process groups

Everything runs on a single 4-core WSL2 machine. The controller is one OS process; the network is Mininet namespaces; the Raft cluster is three further processes that are deliberately NOT connected to the controller yet.

Host — WSL2 · Ubuntu 26.04 · Python 3.14 · 4 cores · no venv, apt packages only

Process 1 — SDN controller

```
sudo python3 -m controller.osken_manager \
controller.trust_balancer
```

listens :6653 OpenFlow 1.3
serves :8081 REST + SSE

threads
FlowMonitor 1 s /status poll
FlowStatsPoller flow counters
PortStatsPoller port counters
ledger commit every 10 updates
NorthboundAPI ThreadingHTTPServer

Process group 2 — Mininet network namespaces (sudo)

core switch s0 — OVS, dpid 0x...01
every VIP task transits it

edge switches s1..s8
OVS · OpenFlow 1.3
5 ms / 1 Gb link to s0

cx — control host
not in a namespace,
bridged to s0

8 × srv node_agent.py
HTTP /status /task

40 × iot iot_client.py
1-10 ms / 10 Mb links

Process group 3 — RAFT cluster

```
python3 -m blockchain.raft_timeline
```

n1 RAFT :9001 control API :9101
n2 RAFT :9002 control API :9102
n3 RAFT :9003 control API :9103

Real OS processes, real loopback TCP,
a real SIGTERM of the leader.

NOT connected to Process 1 — the live
controller still commits single-replica.

Addressing & service model

IoT devices 10.0.0.1 - 10.0.0.40 edge servers 10.0.1.1 - 10.0.1.8
Virtual service IP (VIP) 10.0.99.254 : 9000

A client never addresses a server. It opens a TCP connection to the VIP; the
controller rewrites destination MAC + IP in the data plane (the priority-300
pair), so the routing decision is invisible to the client — and revocable.

Operator access & the control arm

Dashboard http://localhost:8081/ live SSE panels
Offline report http://localhost:8081/analysis built by evaluation/
Flow tables mininet> dpctl dump-flows -O OpenFlow13 | grep cookie=0x5a

Control arm sudo -E python3 -m base_model.run_base --duration 300
same topology, same workload, same attackers, no defences.

Start-up order — enforced by scripts/preflight_live_run.py, because a live run costs sudo and five minutes

1. sudo service openvswitch-switch start
2. sudo modprobe -a sch_htb sch_netem sch_tbf sch_prio ifb
3. sudo mn -c
4. start the controller FIRST — a switch that connects to nothing installs no rules, and the run is silently empty
5. sudo python3 run_demo.py --mode mininet --config config/params_trust_full.yaml
6. python3 -m evaluation.nfr_report data/events.jsonl

Both arms bind :6653 and :8081, so the zero-trust arm and the base_model control arm are run one at a time, back to back on an idle machine.

Figure 4. Deployment view — process groups, ports and the VIP service model.

PROCESS GROUP	WHAT RUNS	PORTS
1 — SDN controller	python3 -m controller.osken_manager controller.trust_balancer	:6653 OpenFlow 1.3 · :8081 REST + SSE
2 — Mininet namespaces	s0 core OVS, s1..s8 edge OVS, 8 × node_agent.py , 40 × iot_client.py, cx control host	VIP 10.0.99.254:9000
3 — RAFT cluster (standalone)	3 × blockchain/raft_replica.py	RAFT :9001-9003 · control API :9101-9103

- Host: WSL2 · Ubuntu 26.04 · **Python 3.14** · 4 cores · apt packages only, no venv and no pip.
- **A client never addresses a server.** It connects to a virtual IP, and the controller rewrites destination MAC and IP in the data plane. The routing decision is therefore invisible to the client and — critically — **revocable** by the controller at any moment, without the client's cooperation.
- Start-up order is enforced by scripts/preflight_live_run.py: the controller must be listening first, because a switch that connects to nothing installs no rules and the run comes out silently empty.

4 How one task flows end to end

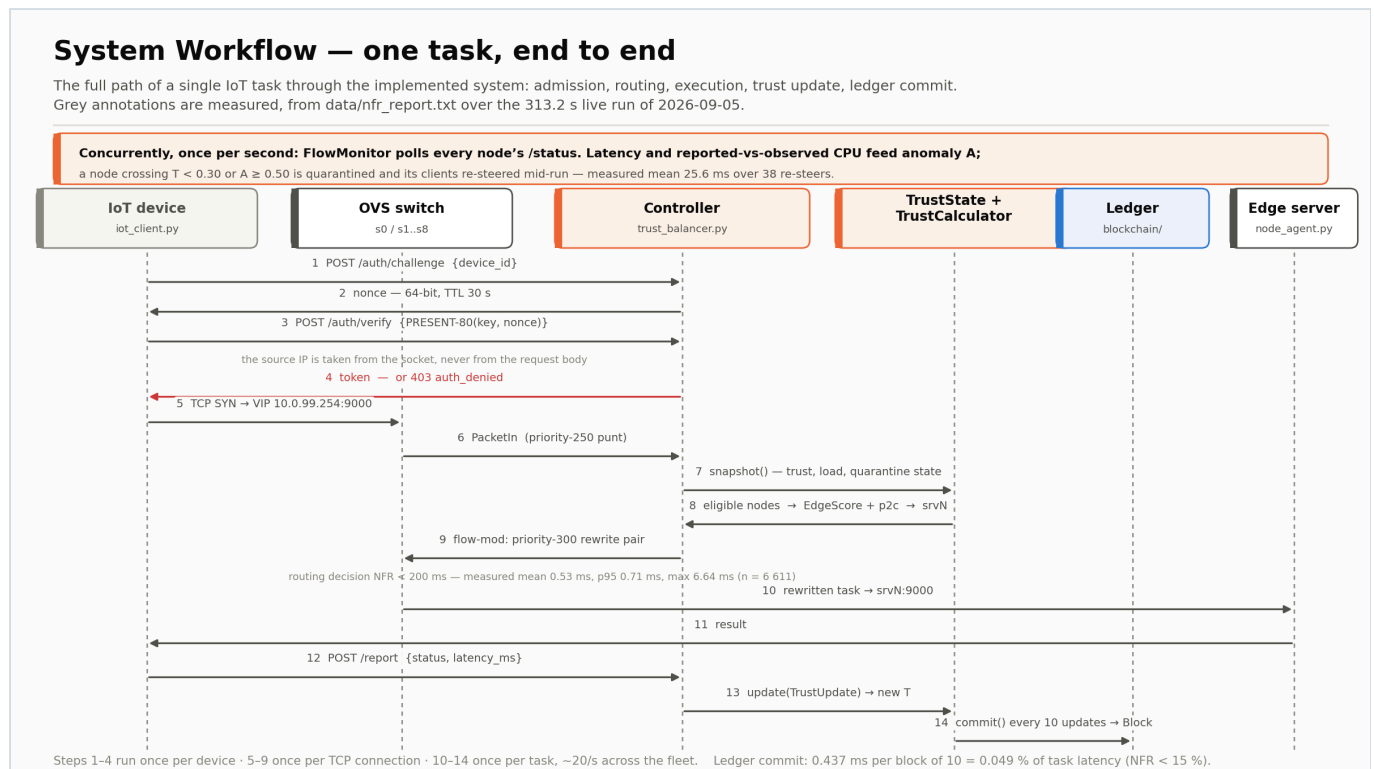


Figure 5. One task from admission to ledger commit. Steps 1–4 happen once per device, 5–9 once per TCP connection, 10–14 once per task — roughly 20 times per second across the fleet.

Phase A — Admission (once per device)

1. The IoT device asks for a challenge: `POST /auth/challenge {device_id}`.
2. The controller returns a fresh 64-bit nonce with a 30-second TTL, single-use.
3. The device encrypts the nonce under the fleet `PRESENT-80` key and returns it to `POST /auth/verify`.
4. The controller checks the cryptographic response **and** the source IP against the provisioned identity roster. Both must pass. A failure returns `403` *and* publishes an `auth_denied` event with a structured reason — a refusal that only the refused device knows about is invisible to any later analysis.

Phase B — Routing (once per TCP connection)

5. The device opens a TCP connection to the virtual IP `10.0.99.254:9000`.
6. The edge switch has no matching flow rule, so it sends a `PacketIn` to the controller.
7. `edge_selector` filters to eligible nodes (trust above the isolation threshold, not quarantined), samples **two** of them, and scores each by `EdgeScore` $w_1 \cdot T + w_2 \cdot (1 - \text{cpu}) + w_3 \cdot (1 - \text{latency})$.
8. The winner is installed as a pair of flow rules that rewrite destination MAC and IP to that specific server — and the reverse rules for the return path.
9. The dispatch is registered in `TrustState`, which now holds an outstanding obligation against that server.

Phase C — Execution, verdict and record (once per task)

10. The server does real bounded work (a hash loop) and answers.

- l1. The client reports the outcome — success or timeout, with its own measured latency — to `POST /report`.
- l2. `TrustState` resolves the dispatch, updates reliability R, latency B and honesty H, and decays the anomaly term A.
- l3. In parallel, the 1-second control loop has been polling every server's `/status`, comparing what it *claims* against what the controller *observed*, and raising anomaly signals.
- l4. The trust update is batched (10 per block) and committed to the ledger; a block carries a Merkle root over its updates and a SHA-256 hash over its header, chained to its predecessor.

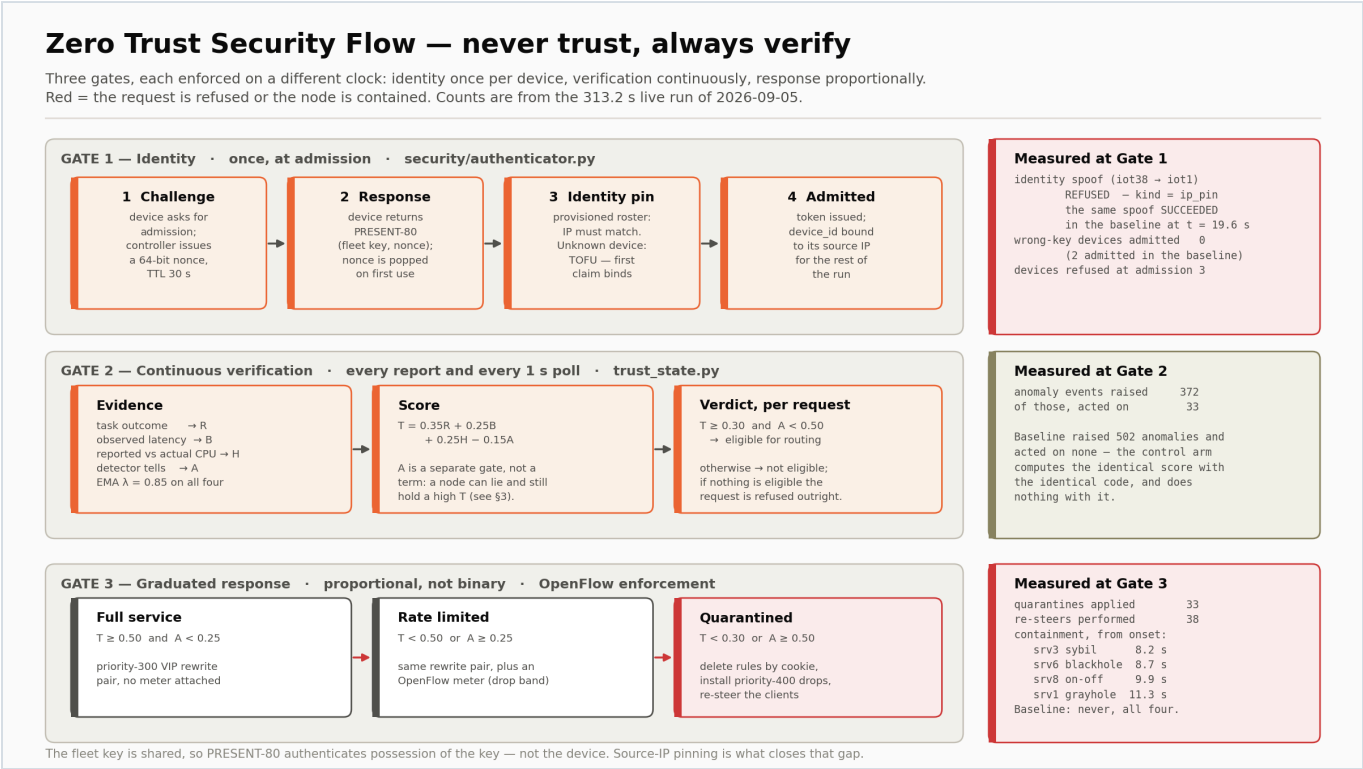
Phase D — Enforcement, when a node fails verification

If trust falls below the isolation threshold, **or** the anomaly gate fires regardless of trust, the controller: deletes every flow rule carrying that node's cookie with a single cookie-matched `OPFPC_DELETE`; installs drop rules at `PRIQ_QUARANTINE_DROP`; and re-steers the in-flight dispatches onto surviving nodes. Measured end to end at **25.6 ms mean, 47.3 ms maximum** against a 3-second requirement.

RECOVERY IS PART OF THE DESIGN, NOT AN AFTERTHOUGHT

A quarantined node is not condemned forever. **Probation** — the half-open leg of a circuit breaker — gives a node that was isolated on the *trust* rail only, and whose anomaly score has since fallen below the gate, one trial task per interval. Trial flows install *above* the quarantine drop rules, or they would be black-holed by the very rules they exist to escape. In the run reported in §7 that produced 23 recoveries from 25 quarantines. §14 explains why this mechanism had to be invented.

5 The trust model, the anomaly gate, and our strongest finding



FINDING 1 — THE PUBLISHED FORMULA PROVABLY CANNOT ISOLATE A COMPETENT LIAR

Replaying the real `TrustCalculator` against the project's own attack scenarios shows that the original claim “ $\bar{A} \rightarrow 1$ implies $T \rightarrow 0$ ” is **arithmetically unreachable**. Consider a node that lies about its load but still serves tasks correctly. Its reliability is perfect and its behaviour term is near-perfect, so:

$$T = 0.35(1.0) + 0.25(0.96) + 0.25(0) - 0.15(1.0) = \mathbf{0.44}$$

δ can only ever subtract 0.15. **Dishonesty alone cannot push the score across an isolation threshold of 0.30**. A perfectly detected, perfectly dishonest node stays eligible for traffic forever.

Consequence: score-based trust is insufficient for Zero Trust's continuous-verification requirement. An independent anomaly gate ($\bar{A} \geq 0.5 \Rightarrow \text{quarantine regardless of } T$) is **necessary, not belt-and-braces**. This is proven in code by `test_f04_non_degrading_liar_needs_anomaly_gate`, which drives the real calculator rather than a mock.

5.2 Why this is a contribution rather than a bug report

We kept the published formula byte-for-byte and added a documented gate beside it. The alternative — quietly retuning the weights until the demo worked — would have destroyed the finding and produced a system whose success nobody could explain. A characterised failure mode with a tested fix is the strongest credibility signal available to a project like this.

The live evidence that the two rails are both necessary comes straight out of the recorded run: the sybil and on-off attackers were caught **only** by the anomaly gate (their trust never fell far enough), while the blackhole attacker was caught **only** by the trust score and raised almost no anomaly signal at all. Neither rail alone catches all four attackers. That single sentence is the justification for the entire two-rail design.

FINDING 2 — A SYBIL LIE IS SELF-DEFEATING WHEN THE TELEMETRY IS REAL

Because our node agents perform real bounded work, a liar that advertises “I am idle” **attracts traffic it cannot actually serve**, genuinely degrades, times out, and its trust collapses on its own. This emergent self-punishment exists *only* because the testbed measures real load instead of injecting synthetic telemetry — which most simulation papers in this space do. The pathological remaining case, a liar with enough spare capacity never to degrade, is precisely the case Finding 1's gate covers. The two findings together are a complete story.

5.3 Load balancing: EdgeScore, and why power-of-two-choices

Secure Load-Balancing Flow — trust as a first-class routing input

The routing decision runs on the first packet of every VIP connection. Trust does not merely re-rank nodes; it removes them from the candidate set entirely, and a quarantine mid-connection tears down rules that are already installed.

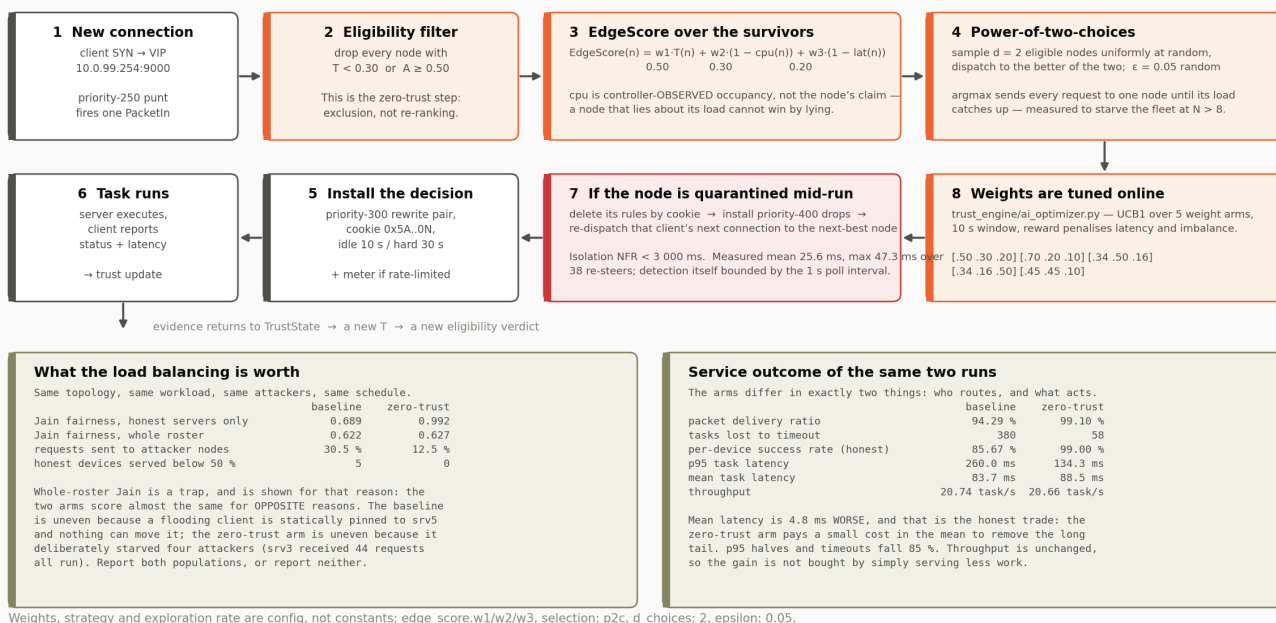


Figure 7. The load-balancing decision: eligibility filter, two-candidate sampling, EdgeScore ranking, flow-rule installation.

The original selector was **argmax** — take the single best-scoring node. We measured what that does as the fleet grows, driving the *real* selector inside a queueing model, and found a genuine flaw:

- With load-only differentiation, argmax spreads traffic fine even at $N=64$ — it degenerates to join-shortest-queue. So “many servers plus argmax” is *not* the problem by itself.
- Once **trust is coupled to selection**, argmax locks onto roughly **two nodes regardless of N** — $N=8 \rightarrow 2$ used, $N=32 \rightarrow 2$ used, $N=64 \rightarrow 3$ used. Positive feedback: only completed tasks move trust, so the nodes already winning traffic are the only ones that can build trust. Idle capacity grows linearly with fleet size.
- Argmax then fails in **two different ways at the two ends of offered load**: at low load it starves the fleet (Jain fairness 0.176, 13 of 16 nodes idle); at high load almost nothing is starved but throughput saturates around 85 tasks/s from $N=8$ upward and delivery ratio collapses from 99.2% to 10.5% at $N=64$.
- **Power-of-two-choices scales linearly**: 48 / 95 / 192 / 382 / 762 tasks/s at $N=4/8/16/32/64$, delivery ratio 99.7–99.8%, delay flat at ~266 ms.

Do not claim p2c as a CPU saving. We measured that too, and it is not one: both strategies cost the same 3/5/9/16/30 μ s per decision, because the selector scores and sorts every eligible node either way — the full ranking is returned as the decision's explanation and published to the dashboard. That is a deliberate trade. Claim the p2c win in fairness, throughput and delay; never in CPU.

6 The six attacks and how each is caught

All six have **delayed onset**. Before it arms, an attacker is behaviourally identical to an honest node — so every attack has a clean baseline interval before it, and the detectors have to notice a change rather than recognise a permanently odd node.

ATTACK	SUBJECT	ONSET	WHAT IT DOES	HOW IT IS CAUGHT
Sybil (load lying)	srv3	20 s	Claims <code>cpu ≈ 0</code> while burning CPU, to attract traffic it cannot serve	Latency tell — it answers <code>/status</code> slowly no matter how little task traffic it receives
Blackhole	srv6	30 s	Accepts tasks and completes none; still answers <code>/status</code> at full speed	Timeout-rate tell , and the trust rail collapsing on reliability
Grayhole	srv1	40 s	Drops 50% of tasks — partial, so it stays plausible	Same timeout-rate tell, weaker per sample; separated from blackhole on the median rate over a window
On-off	srv8	50 s	Alternates good and bad phases (20 s period, 0.5 duty) to bleed trust back up between attacks	Latency tell plus sustained intermittency — the share of the active span spent quiet
DDoS / flood	iot37	60 s	3 workers at 0.25 s $\approx$ 12 Hz, 24× the honest per-device rate	<code>flood_detector.py</code> , keyed on the requesting client
Identity spoof	iot38	15 s	Holds the real fleet key and impersonates iot1	Source-IP pin against the provisioned roster → <code>auth_denied kind=ip_pin</code>
<i>Bad credentials</i>	iot39, iot40	—	Wrong key entirely — the seventh ground-truth class	Refused at admission → <code>auth_denied kind=bad_response</code>

6.1 Three design decisions worth defending out loud

1. **The flood tell is keyed on the requesting client, never on the edge node it lands on.** A server drowning under a flood is not lying about its load — it is telling the truth about being overwhelmed. Blaming the node would repeat a mistake this project has already made twice (§14).
2. **We did not build a literal replay attack.** `verify_response` pops the nonce on first use, so replay was already structurally impossible. Building a fake version of an attack the system already defeats would have been dishonest. The real gap was that the key is **fleet-wide**, so PRESENT-80 authenticates *possession of the key*, not *the device* — which is exactly what source-IP pinning closes.
3. **node_down is deliberately not one of the six attacks.** Our blackhole answers `/status` fine and drops only tasks, so “silent” is a liveness failure, not an attack. Folding it in would inflate the very numbers the classifier exists to measure honestly.

6.2 Naming an attack is harder than detecting it — and we say where the limits are

Two of the six pairs are **not separable from a single detection cycle**, so the classifier reads an evidence *window*:

- **Blackhole vs grayhole** raise the same signal and differ only in **magnitude**.
- **Sybil vs on-off** raise the same signals and differ only in **intermittency**.

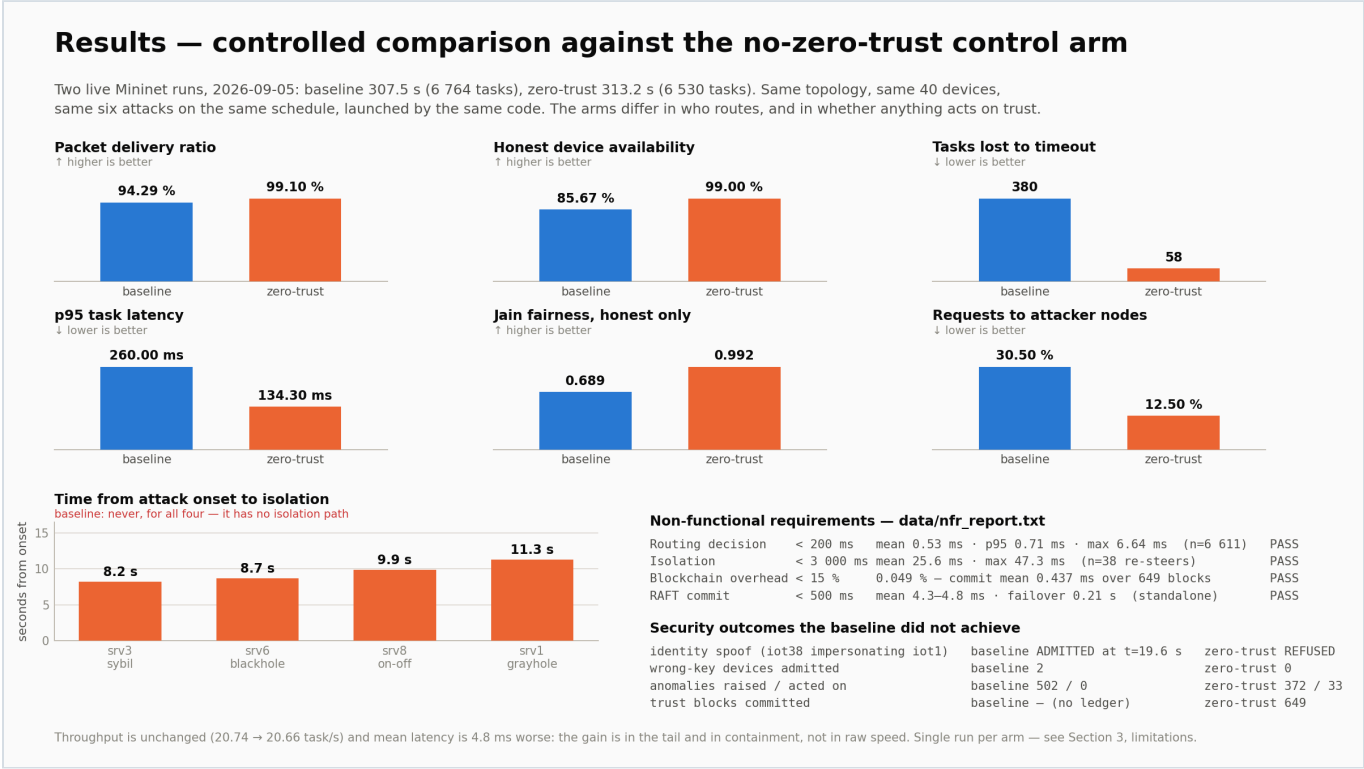
THE TENSION WE MEASURED, AND COULD NOT TUNE AWAY

Containment quality and magnitude evidence are in direct opposition. A blackhole fails everything, so it is isolated fastest, so it produces the fewest outcomes with which to characterise it. In one run srv6 produced 3 task outcomes in 312 s; the grayhole, which kept partly working, stayed in circulation and produced 24. The faster the defence, the thinner the evidence for the *label*.

Our resolution: when no measured rate exists, the trust rail establishes the **family** and the call degrades to the *weaker* claim (grayhole), never the stronger one. **In every such case the attacker is still caught and still quarantined — only the label degrades.** That is the honest framing, and it is the one to use if a panellist asks about the two misclassifications in §7.

7 Results: the controlled comparison

Two live runs on 2026-09-05: the same topology, the same 40 devices, the same six attacks on the same schedule, launched by the same code. The arms differ in exactly two things — **who decides where a connection goes**, and **what happens when a node misbehaves**. This is the single most important slide in the review.



METRIC	BASELINE	ZERO-TRUST	READING
Packet delivery ratio	94.29 %	99.10 %	Fewer tasks lost outright
Mean per-device availability (honest devices)	85.67 %	99.00 %	The service the honest fleet actually received
Honest devices served below 50 %	5	0	Nobody is left starving behind a bad server
Tasks lost to timeout	380	58	–85 %
p95 task latency	260.0 ms	134.3 ms	The tail halves
Mean task latency	83.7 ms	88.5 ms	We lose here by 4.8 ms — see the note below
Throughput	20.74 task/s	20.66 task/s	Unchanged — the gain is not bought by serving less work
Jain fairness, honest servers only	0.689	0.992	Near-perfect balance across the servers that deserve traffic
Requests routed to attacker nodes	30.5 %	12.5 %	Less than half as much work handed to attackers
Anomalies raised / acted on	502 / 0	372 / 33	The control arm's cost: it detects and can do nothing
Re-steers performed	0	38	In-flight work rescued off failing nodes
Identity spoof (iot38 → iot1)	ADMITTED at t=19.6 s	REFUSED (ip_pin)	The attacker ran 130 tasks under a stolen identity in the baseline
Wrong-key devices admitted	2	0	No admission control at all in the baseline
Trust blocks committed	— (no ledger)	649	The audit trail

STATE THE LOSS BEFORE A PANELLIST FINDS IT

Mean latency is 4.8 ms worse in our system. That is the trade, and it is a good one: we pay a small cost in the mean to remove the long tail. p95 halves, timeouts fall 85%, and throughput is unchanged — so the improvement is not bought by quietly serving less work. Volunteering this number is worth more than the 4.8 ms costs.

7.1 Containment: time from attack onset to isolation

SERVER	ATTACK	BASILINE	ZERO-TRUST	REQUIREMENT
srv3	Sybil	never	8.2 s	Isolation re-dispatch NFR < 3 000 ms once the decision is made; detection itself is bounded by the 1 s poll interval
srv6	Blackhole	never	8.7 s	
srv8	On-off	never	9.9 s	
srv1	Grayhole	never	11.3 s	

“Never” is not zero — it is the absence of a mechanism. The baseline has no quarantine path, no re-steer path, no admission control and no ledger. It raised 502 anomalies and acted on none of them. That is precisely the cost the experiment was built to price.

7.2 Trust separation per server

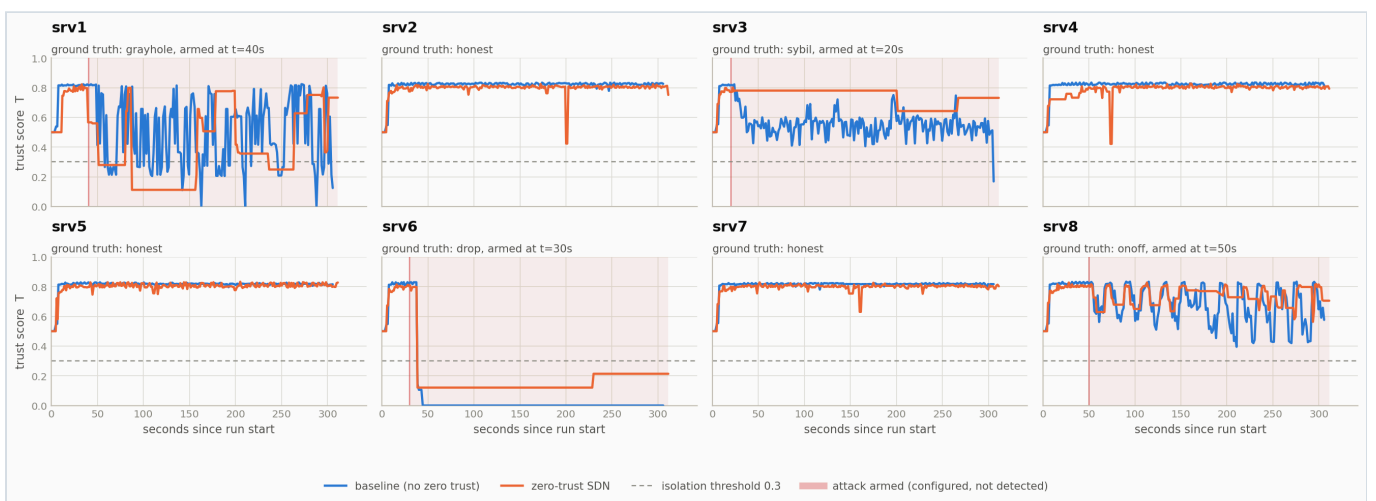


Figure 10. Trust trajectories per server in both arms. Read per node, never as a fleet mean — averaging a blackhole's collapse against seven healthy servers produces a gentle dip and hides the finding entirely.

Two trajectories in the baseline arm are the paper's figures. The blackhole srv6 flatlines at trust **0.000 for 263 seconds** and is *never* isolated, because nothing in the control arm acts on trust. And the on-off attacker srv8 — mean trust 0.677 — **never crosses the isolation threshold at all**, which is the on-off attack working exactly as the literature says it does. Meanwhile **zero** honest servers were falsely accused in either arm.

7.3 Load distribution

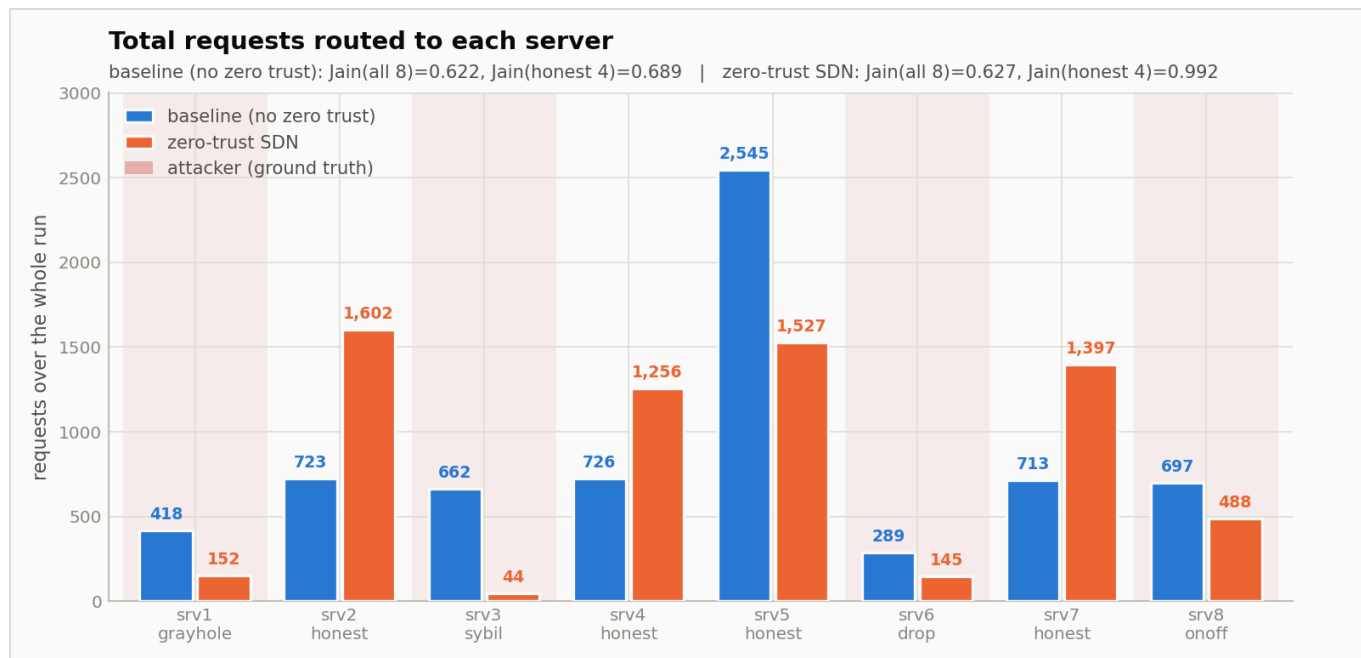


Figure 11. Share of routed requests per server in both arms.

A METRIC THAT LIES, AND HOW WE HANDLE IT

Jain fairness over the **whole roster** is 0.622 baseline versus 0.627 zero-trust — almost identical, and reported alone it reads as “the load balancer made no difference”. The two arms are uneven **for opposite reasons**: the baseline is uneven because a flooding client is statically pinned to one server and nothing can move it; our arm is uneven because it *deliberately starved four attackers*. Restricted to the honest servers the figure is **0.689 versus 0.992**. We therefore always report two populations — full roster and honest-only — plus the attacker share. If a panellist quotes the whole-roster number back at you, this is the answer.

7.4 The statistical evidence — 600 seeded runs

Live runs give fidelity; they do not give confidence intervals. A separate harness drives the **real** selector, the **real** trust calculator and both shipped detectors inside a seeded discrete-event simulation: 5 strategies × 4 scenarios × 30 seeds = **600 runs**, paired on common random numbers, tested with the paired **Wilcoxon signed-rank** test and corrected with **Holm-Bonferroni**.

SCENARIO	VS LEAST-CONN	VS RANDOM	VS ROUND-ROBIN	VS NO-TRUST (ABLATION)
Sybil attacker	−64.6 %	−29.6 %	−19.7 %	−15.1 %
Dropping attacker	−61.8 %	−49.4 %	−48.7 %	−25.3 %
Both attackers	−68.8 %	−52.3 %	−48.5 %	−29.8 %

Reduction in SLO-violation rate, all at $p_{\text{adj}} = 7.45 \times 10^{-9}$ with rank-biserial $r = +1.00$ — meaning our system won in **30 of 30 paired seeds**. Failure rate falls 97.7% against the strongest baseline and 99.2–99.3% against the trust-blind ones. **14 of 16 comparisons significantly favour the system**.

The no_trust column is the one that matters. It is the identical selector with the trust term removed and quarantine disabled — so it attributes the improvement to *trust*, not to load-aware routing in general.

THE TWO COMPARISONS WE LOSE, AND WHY WE REPORT THEM

Both losses are in the **clean** scenario, with no attacker present: we are 10.7% worse than round-robin and 4.8% worse than random. That is a genuine cost of a load-reactive strategy and we report it as one. The cause is **not** the trust term — it is staleness in the load signal that *every* load-reactive strategy reads. A poll-interval sweep isolates it exactly: halving the poll interval from 1.00 s to 0.50 s closes most of the gap. Being able to name the mechanism behind your own loss is worth more in a viva than not having the loss.

8 Testing, NFRs and consensus evidence

8.1 Non-functional requirements

NFR	TARGET	MEASURED	VERDICT
Routing decision (packet-in → flow-mod)	< 200 ms	mean 0.53 ms · p95 0.71 ms · max 6.64 ms (n = 6 611)	PASS
Isolation (quarantine → re-dispatch)	< 3 000 ms	mean 25.6 ms · max 47.3 ms (n = 38)	PASS
Blockchain overhead	< 15 %	0.049 % — commit mean 0.437 ms over 649 blocks, against 88.5 ms mean end-to-end task latency	PASS
RAFT commit latency	< 500 ms	mean 4.3–4.8 ms ; failover 0.21 s (standalone 3-replica cluster)	PASS

THE DENOMINATOR TRAP — KNOW THIS ONE, IT IS A CLASSIC VIVA QUESTION

An early version of our own harness divided ledger commit cost by the handling time of a single uncommitted request (tens of microseconds) and reported a 0.424 ms commit as **2 155 % overhead**. The correct framing amortises the commit over the batch size and divides by **end-to-end task latency** — the thing the user actually waits for — which gives 0.049 %. If someone challenges the “blockchain overhead” number, this is the reasoning to give.

8.2 Test suite

Preliminary Testing — 1 004 tests, 993 passing, 11 skipped, 0 failing

The suite is not a coverage number. Most of these tests exist because a specific defect reached a live run first; each one now fails loudly if that defect is reintroduced. Run: `python3 -m pytest -q` (50.7 s on the 4-core box).

Unit — pure logic, no I/O

test_raft.py R-01..R-18, five safety properties, virtual clock
test_attack_classifier 32+ · both resolution limits
test_trust_state.py dispatch accounting, probation
test_flood_detector.py 9 · the per-client tell
test_blockchain.py B-02 tamper detection
test_authenticator.py nonce reuse, IP pin, TOFU
test_plot_raft_timeline 10 · the failover analysis

Integration — real sockets, real processes

test_commit_backend.py 3-replica cluster over TCP;
kill the leader, keep committing
test_raft_replica.py the HTTP control API
test_iot_client_spoof denial stops traffic, success
sends it, delayed onset honoured
test_node_agent.py grayhole, on-off, delayed onset
test_live_config_prepl. 12+ · real launcher vs real
argparse, before a live run

Parity — the control arm must match

test_base_config_parity every shared key identical
test_observer_parity load_window_s pinned to the
treatment default (3.0 s)
test_launcher_parity both arms launch agents
through the same function
test_compare.py a spoofed identity is broken
out, never silently merged
test_plot_load.py whole-roster Jain hides it

Tests that exist so a future change fails loudly

Each encodes a defect that reached a live run before it was caught, and each states the rule that fixed it:

- a detector with no recent evidence ABSTAINS — it never re-asserts a stale verdict (quarantine had become an absorbing state)
- `sum(_inflight) == len(_dispatches)`, the occupancy invariant (a register leak was fabricating load)
- a re-steered dispatch is not chargeable to the node it was torn from (nodes were charged for tasks never received)
- “correctly isolated attacker” and “wrongly quarantined honest node” never share a metric (averaging hides whichever one matters)
- task-level loss and OpenFlow drop-rule hits never share a series
- the dashboard recomputes the chain hash in the browser rather than trusting the controller’s own valid flag

Known limitations, stated rather than hidden: one run per configuration (no confidence intervals) · detection latency is an upper bound, bounded by the 1 s poll · scalability past 8 nodes is a queueing simulation, not a live run · DDoS is detected and classified, but not throttled.

Live failover test — recorded, not asserted

blockchain/raft_timeline.py runs the 3-replica cluster under continuous load, SIGTERMs the leader mid-run and records every role change and every commit attempt; evaluation/plot_raft_timeline.py draws it.

Measured, 45 s run of 2026-09-07:

commits attempted 853
commits succeeded 850 (99.6 %)
commit latency mean 2.6 ms · max 19.4 ms
leader failover 0.21 s (± 0.05 s poll)
service gap 0.21 s
attempts refused during it 3
commit cost, 3/3 → 2/3 → 3/3 2.23 → 3.08 → 2.58 ms

The restarted replica rejoins with an EMPTY log and is caught up by the leader — crash-fault only, no disk persistence. That limit is on the figure, not in a footnote.

Live runs are excluded from the suite by design — one costs sudo and five minutes. scripts/preflight_live_run.py checks the config against the real launcher before a run is started.

Figure 9. Test evidence: composition of the suite and what each family pins.

1 004 tests collected — 993 passing, 11 skipped, 0 failing, in 50.7 s. Three families:

- **Unit** — pure logic, no I/O: the classifier, the trust formula, the Merkle tree, the cipher against published test vectors.
- **Integration** — real sockets, real processes, real SIGTERM: the spoof is verified end-to-end over a genuine socket against the real /auth/verify, and the RAFT leader is genuinely killed.
- **Parity** — the control arm must match the treatment arm key for key, so the comparison in §7 stays controlled.

A category worth mentioning: tests that exist so that a *future* change fails loudly. Examples include a test asserting that our re-steer attribution rule reads no clock (so nobody can slip a tunable threshold into it), a three-way pin on the quarantine drop-rule priority, and a preflight assertion that the identity roster is actually **populated** rather than merely harmless — which caught a fix that would otherwise have shipped into a live run silently doing nothing.

8.3 Consensus: what RAFT does and does not do here



Figure 12. A recorded 45-second RAFT run: role lanes, term, chain length and commit latency, with the leader killed at 15 s and restarted at 30 s. Three real processes over real TCP, under continuous load, with the leader killed by a genuine signal:

- Leader failover in **0.21 s**; 850 of 853 commits succeeded (**99.6 %**).
- The outage is **three refused attempts** — visible only because the client never pauses for the kill.
- A degraded 2-of-3 cluster costs more per commit (2.23 → 3.08 → 2.58 ms after rejoin). We report the observation; we have **not** established the cause and do not claim one.
- A restarted replica rejoins with an empty log and is caught up by the leader — making the crash-fault limitation visible rather than merely documented.

SAY THIS PLAINLY, BEFORE YOU ARE ASKED

“RAFT is implemented, unit-proven for safety, and demonstrated live over real TCP with a leader kill. **The controller still commits single-replica**, and the swap is deliberately deferred.” Every block in every live run carries `raft_term = 0` and `proposer_id = "controller"`. Three concrete runtime blockers stand in the way, all identified: nothing drives the election tick inside the controller; `commit()` would block an OpenFlow handler thread for up to 2 s; and a `None` return is currently discarded silently, which would lose trust updates on every leadership change. **Never let a slide imply the live runs are RAFT-backed.**

9 Where this is used — real-world application domains

This is the section panellists most often push on, and the trap is breadth. A report that maps its architecture onto **one** real domain reads as engineering; one that lists eight domains reads as marketing. Lead with MEC, keep industrial IoT as the secondary, and have the rest ready only if asked.

9.1 Strong fit — claim these

① Multi-access Edge Computing (MEC) and the 5G telco edge PRIMARY

Telecom operators run third-party workloads on distributed edge sites — thousands of small compute nodes at base stations, central offices and aggregation points. The operator genuinely cannot fully trust every node in that estate: some are in unmanned cabinets, some run tenant software, some are maintained by contractors. And MEC orchestrators route work by **self-reported host load** today. The mapping onto our architecture is almost one-to-one:

OUR COMPONENT	MEC EQUIVALENT
SDN controller	MEC orchestrator / MEC platform manager
Edge servers (srv1..srv8)	MEC hosts running tenant applications
IoT clients (iot1..iot40)	User equipment and connected devices
VIP + flow rewrite	Service-level traffic steering at the user-plane function
Trust score + anomaly gate	Continuous host attestation with routing consequences
Tamper-evident ledger	Auditable record for multi-tenant SLA disputes

The pitch sentence: “An operator can host a tenant's workload on a node it does not fully control, and still bound its exposure: if the node starts lying or dropping, traffic is revoked from it in under a second and there is a signed record of why.”

② Industrial IoT and the factory / OT edge SECONDARY

Operational-technology networks increasingly use SDN-style central control. The scenario that keeps OT engineers awake is not a machine that stops — that is obvious and alarms fire — it is **a controller that keeps “working” while lying about its state**, because every safety and scheduling decision downstream is computed from that state. Our anomaly gate is aimed at exactly that case: quarantine on *behaviour*, not merely on a score. And the sub-3-second bounded isolation is the kind of number OT people ask for first, because it maps onto a process safety time.

③ Smart campus and smart-city gateways

Many small operators, physically accessible cabinets on poles and in kerbside enclosures, and no dedicated security operations centre watching them. The value here is **unstaffed autonomous operation**: a control plane that detects, contains and re-steers by itself at three in the morning, and leaves an audit trail a municipal operator can hand to an auditor afterwards. This is the single clearest “nobody is watching” story in the list.

9.2 Plausible fit — mention only with the caveat attached

DOMAIN	WHY IT FITS	THE CAVEAT YOU MUST STATE
Federated / community edge (volunteer nodes, CDN-lite)	Nodes owned by strangers is exactly where behavioural trust matters most	The controller itself becomes the trust bottleneck — who runs it? We do not address that.
Tactical / disconnected operations	Continuous verification with local enforcement suits contested environments; replicated state suits partition tolerance	Our ledger is crash-fault tolerant, not Byzantine. In a contested environment that distinction matters more, not less.
Multi-tenant private edge clouds	An operator hosting several tenants' workloads on shared hardware needs evidence-based, not declaration-based, placement	Requires per-tenant identity provisioning and an OpenFlow-capable or programmable data plane.

9.3 Weak fit — do not claim these, and say why if asked

DOMAIN	WHY WE DO NOT CLAIM IT
Public cloud data centres	Hyperscalers own and trust their own hardware, run attested boot, and have vastly richer telemetry than a 1 Hz status poll. The problem we solve is largely not their problem.
General internet / inter-domain routing	There is no single control plane. Trust between operators is a contractual and reputational problem, not an SDN one.
Consumer IoT security as a product	“Secure your smart home with blockchain” is exactly the buzzword framing this project is structured to avoid. Our mechanisms target infrastructure operators, not consumers. Stretching it there would invite hostile questions and deserve them.

IF YOU ARE ASKED FOR ONE SENTENCE ON APPLICABILITY

“Anywhere a central control plane must place work on compute nodes it does not fully control, and where the consequence of trusting a node's self-report is a security problem rather than merely an efficiency one. That is MEC and the telco edge first, industrial and municipal edge second — and explicitly *not* the public cloud, where the operator already owns the hardware.”

10 Systems this maps onto

A different and very effective way to answer “where is this used?” is to name the systems a panellist already knows, and say precisely what this mechanism would change about each. These are **architectural analogies**, not integrations we built — say so, and the analogy becomes a strength rather than an overclaim.

EXISTING SYSTEM	HOW IT DECIDES TODAY	WHAT THIS PROJECT WOULD CHANGE
Kubernetes scheduler / autoscaler	Places pods using resource metrics reported by the kubelet on each node , plus liveness and readiness probes that the node itself answers.	A node score derived from <i>observed task outcomes</i> rather than self-reports; <i>cordon / taint</i> becomes the equivalent of our quarantine, applied automatically on behavioural anomaly rather than on a failed binary probe.
Service mesh outlier detection (Envoy / Istio)	Ejects a host from the load-balancing pool after consecutive 5xx responses or gateway failures.	Outlier detection is blind to our central case: a host that returns perfectly good 200 s while lying about capacity is invisible to it. Our honesty term and latency tell are exactly the signal it lacks.
Cloud load-balancer health checks (ELB, GCLB, HAProxy)	A periodic probe with a binary pass/fail verdict, answered by the backend.	Replaces a binary self-answered probe with a graded, continuously-updated score computed by the balancer from evidence the backend cannot fabricate , plus an independent gate that can act when the score cannot.
ETSI MEC orchestrator	Places applications using advertised host capacity and declared capabilities.	Trust-weighted placement, and — the part that matters — revocation in the data plane rather than only at the next placement decision.
SD-WAN controller	Chooses paths using measured link SLA — loss, latency, jitter — that the controller observes itself.	This is the closest existing analogue, and a useful one: SD-WAN already trusts <i>measurement over declaration</i> , but applies it to links . We apply the same discipline to compute nodes , where declaration is still the norm.
CDN / edge cache request routing	Steers by geography, capacity declarations and health signals from the edge PoP.	For federated or partner-operated PoPs, behavioural trust bounds what a misbehaving PoP can absorb before it is cut out. (Caveat from §9.2 applies.)
NIST SP 800-207 Zero Trust Architecture	Defines a Policy Decision Point and Policy Enforcement Point, and mandates continuous verification — but leaves the mechanism open.	Our controller <i>is</i> a PDP and the OpenFlow rules <i>are</i> the PEP. The contribution is applying the model to the serving infrastructure , where most implementations apply it only to users and client devices.

THE STRONGEST ONE-LINE FRAMING IN THIS WHOLE DOCUMENT

“SD-WAN already decided that **measurement beats declaration** — for network links. Nobody has finished doing the same thing for compute nodes. That is the gap this project works in.”

10.1 What a real deployment would require

If a panellist asks “could you actually deploy this tomorrow?”, the honest answer is “not tomorrow, and here is exactly what stands between”:

1. An **OpenFlow 1.3-capable or otherwise programmable data plane** (P4, eBPF/XDP, or a smart NIC). The enforcement mechanism is what makes the verdict binding; without it the trust score is advisory only.
2. **Per-device identity provisioning** — our source-IP pin is derived from a provisioned roster. Real deployments would want per-device keys and a 128-bit primitive behind the same Authenticator seam (§13).
3. **Controller redundancy.** The controller is currently a trusted single point. Wiring in the RAFT cluster we already built is the first step; full controller distribution is future work.
4. **Multi-host validation.** Everything measured here runs on one machine — real processes and real sockets, but not real network separation.

11 The business and operations case

Panels frequently ask a version of “so what?” that is not technical. This section is the answer in the language of whoever pays for the system.

11.1 What the operator actually gains

CONCERN	WITHOUT THIS SYSTEM	WITH IT
Mean time to contain	A misbehaving edge node is contained when a human notices — hours, if there is a SOC; never, if there is not. Our control arm measured <i>never</i> , across a full run, with 502 anomalies visible the whole time.	8-12 seconds , unattended, for all four server-side attacks.
Blast radius	Five of forty honest devices dropped below 50% service because they were statically bound to a bad server and nothing could move them.	Zero devices below 50%. Work is re-steered off a failing node while still in flight.
Wasted capacity	30.5% of all requests were handed to attacker nodes.	12.5%, and falling as the detectors converge.
Audit and dispute	No record of why anything was routed anywhere.	A hash-chained, Merkle-proven ledger of every trust decision — 649 blocks in a 313-second run — that a third party can verify without trusting our controller's own claim that it is valid.
Staffing	Containment requires someone watching.	Containment is autonomous; humans read the ledger afterwards.

11.2 What it costs

- **4.8 ms of mean latency** — measured, stated in §7, and bought back several times over in the tail.
- **0.049% ledger overhead** against end-to-end task latency.
- **Sub-millisecond routing decisions** (0.53 ms mean) — the control plane is not the bottleneck.
- **A programmable data plane**, which most greenfield edge deployments already have and most brownfield ones do not.

11.3 Why the ledger is not decoration

This is worth rehearsing because “blockchain” invites cynicism, and the cynicism is usually earned. Our defensible position, stated in one paragraph:

The ledger provides **tamper-evidence** — hash-chained, Merkle-proven history, so retroactive falsification of trust records is *detectable and localisable to the seam* — and **availability** through replication, within a controller cluster that is itself inside the trusted computing base. It is **not** a permissionless blockchain, it does **not** tolerate Byzantine replicas, and it does not need to: our threat model puts malicious behaviour at the **edge nodes**, not inside the controller cluster.

And when the inevitable follow-up comes — *“then why not just use a signed append-only log?”* — the honest answer is: **a hash-chained Merkle log replicated by RAFT essentially is one**, structured for per-record proofs and batch commits. “Private blockchain” is the deck's name for it. Conceding that gracefully costs nothing; defending the buzzword costs credibility.

12 What is genuinely novel here

Three findings, none of which came from the literature review. All three came from building the thing and measuring it.

Finding 1 — the published trust formula cannot isolate a competent liar

Covered in full in §5. The short version: a node that lies about its load while serving correctly floors at $T = 0.44$ against an isolation threshold of 0.30, so a perfectly-detected liar remains eligible for traffic forever. δ can subtract at most 0.15. The consequence is that an independent anomaly gate is **necessary**, not optional — and the live run confirms it, with two attackers caught only by the gate and one caught only by the score.

Why it is a contribution: we found the published formula insufficient, characterised exactly why, kept the formula intact, and added a documented gate. Quietly patching the weights would have destroyed the result.

Finding 2 — a Sybil lie is self-defeating when telemetry is real

Because the node agents perform real bounded work, a liar advertising “I am idle” attracts traffic it cannot serve, degrades, times out, and its trust collapses on its own. This emergent self-punishment **cannot be produced by injected telemetry** — and injected telemetry is what most simulation papers in this space use. The remaining pathological case (a liar with spare capacity that never degrades) is precisely the case Finding 1's gate covers.

Finding 3 — honest packet visualisation in SDN is structurally hard

Once a flow rule is installed, matching packets are forwarded entirely in the data plane and **never reach the controller again**. A controller therefore *cannot* honestly animate per-packet flow from PacketIn events. Our dashboard drives its animation from 1 Hz `OFPFLOWStatsRequest` counter deltas and prints the measured packets-per-second on screen so the claim is checkable. It is a small but real observation about SDN observability — and it is the confident answer to “why doesn't your dashboard show every packet?”

Supporting contributions

- **Trust decisions enforced in the data plane** — VIP rewrite plus cookie-scoped rule deletion on trust collapse, inside a bounded isolation NFR — rather than simulated in a scheduler.
- **“Unreachable equals anomalous”** — an unresponsive status endpoint is itself an anomaly signal. This started as a bug caught by writing tests and became a design rule, with a companion rule that a node which has *never* been seen is `UNKNOWN`, not anomalous.
- **A control arm, not a claim.** Building a complete no-zero-trust version of the same system, launched by the same code, is what makes §7 an experiment. Most projects at this level compare against a description of a baseline rather than a running one.
- **Running a full SDN research stack on Python 3.14 / os-ken** with a hand-written application manager, where the standard tooling does not exist.

13 Engineering decisions we would defend

DECISION	WHY
Power-of-two-choices over argmax	Argmax is winner-take-all and, once trust is coupled to selection, locks onto ~ 2 nodes regardless of fleet size. Measured to starve the fleet as N grows; p2c scales linearly. §5.3.
The anomaly gate is a separate gate, not a fourth term in the formula	A node that lies but serves well holds $T \approx 0.44$ and never crosses the isolation line. Adding another weighted term cannot fix an arithmetic ceiling; a gate can. §5.1.
A detector with no recent evidence abstains rather than re-asserting its last verdict	Without this, quarantine became an absorbing state : isolation cuts traffic, traffic is the only thing that produces outcomes, so every detector went blind the instant it fired and the node could never be exonerated. §14.
The flood tell is keyed on the requesting client, never the edge node	A server drowning under a flood is telling the truth about being overwhelmed. Blaming it repeats a mistake this project already made twice.
No literal replay attack was built	Nonces are popped on first verification, so replay was already structurally impossible. Building a fake version of a defeated attack would have been dishonest. The real gap was fleet-wide key sharing, which source-IP pinning closes.
PRESENT-80 framed as lightweight-crypto engineering, not as strong security	80-bit keys are below modern margins and PRESENT-80 has published cryptanalysis. The honest framing: an ISO/IEC 29192-2 lightweight cipher appropriate to severely constrained devices, demonstrating the challenge-response <i>architecture</i> ; a production deployment would drop a 128-bit primitive into the same Authenticator seam. The seam already exists — which converts a weakness into evidence of good architecture.
Re-steered work is not charged to the node that inherits it	When quarantine tears down in-flight TCP connections, the client blocks on a dead socket until its own timeout — and that timeout was being charged to whichever node inherited the dispatch. An honest node was quarantined twice for tasks it never received. The rule that fixed it reads no clock : a flow was re-steered or it was not, so there is no threshold anyone can quietly mistune later.
The scalability sweep is kept off the dashboard	On a page where every other panel reports the running system, a projection five fleet sizes past anything ever instantiated is not defensible under questioning. The sweep lives in CLI output, where it reads as the simulation result it is.
Structural typing for the seams	<code>CommitBackend</code> , <code>Transport</code> and <code>Authenticator</code> are protocols: <code>RaftBackend</code> is a drop-in for <code>LocalLedgerBackend</code> , and the RAFT core did not change at all to gain a TCP transport.

14 The eleven-run story — what broke and what it taught

This is optional material for the panel, but it is the most persuasive thing in the project if the conversation turns to engineering process. The first full-scale live run **collapsed completely**. Every subsequent fix came from measuring a recording rather than from guessing.

RUN	WHAT WENT WRONG	WHAT IT TAUGHT, AND WHAT SHIPPED
1	All 8 servers quarantined, 91.7% of routes denied. Looked like a total detection failure.	It was not. The three real attackers were caught first and fast ; the honest nodes fell afterwards absorbing re-dispatched load. Two bookkeeping defects: a connectivity test ran before the agents started (so nodes were scored for not answering on processes that did not exist), and the controller's stale-dispatch reaper ran 15× longer than the client timeout, manufacturing phantom load.
2	Still collapsed, 76.1% denial.	Re-steering was resetting the client's abandonment timer, so dispatches were rejuvenated exactly when they needed to age out. And the controller was scoring nodes that had never existed — a node that has never answered is now UNKNOWN, not anomalous.
3	Clean start, then everything quarantined again — via a completely different cause.	Quarantine was an absorbing state. Isolation cuts traffic; traffic is the only thing that produces task outcomes; so every outcome-reading detector went blind the instant it fired, and the system had no evidence-generating path out. Fixes: stale-evidence abstention , and probation — one trial task per interval for a node isolated on trust alone.
4	707 quarantines of a perfectly healthy server.	A dispatch-registration leak fabricated occupancy that never decremented. Little's Law was the diagnostic : predicted in-flight 0.01 against an actual counter of 2 proved the load was fabricated, not real — and killed a plausible wrong hypothesis that would have cost headroom and fixed nothing.
5	—	First clean full-scale run. Zero routing denials, 99.87% task success. Confirmed the fix on the same measurement that found the bug.
7	48% honest availability, 22.5% total outage.	The honesty check's fallback path produced 100% of the false quarantines — the fixed path produced none. The lesson that generalises: replaying a fixed recording measures only first-order effects . A feedback loop is invisible to replay by construction, so a replay estimate is a lower bound and only a re-run settles it.
8	—	Honest availability 48% → 98.9% , route denial 28.3% → 0% , outage → none. Cutting one link collapsed the entire cascade.
9	An attack got in. The spoofer authenticated as its victim and ran 130 tasks under the stolen identity.	Three individually-minor issues composed into a real vulnerability: a stdlib accept-queue of 5 against 48 hosts connecting at once, a client that treated a connection reset as a denial, and trust-on-first-use identity pinning. A reset means the question was never asked; a 403 means the answer was no. Fixed all three.
10	One honest server still mislabelled.	8/8 attacks classified correctly, first time. And an explicit non-claim we wrote down: the victim had already claimed its identity pin before the spoof attempt, so plain TOFU would have refused this spoof too — the new roster was not load-bearing in this run . Saying “8/8, therefore the roster works” would claim more than the run shows.
11	—	100% honest availability, 0/40 honest mislabels, 4/4 honest nodes never quarantined, 0% route denial, 100% of the run above quorum. The re-steer attribution fix fired live for the first time, and we validated it from the run's own counterfactual — recharging the withheld outcomes shows one honest server's

RUN	WHAT WENT WRONG	WHAT IT TAUGHT, AND WHAT SHIPPED
		timeout rate would have gone 0.00 → 0.50, while both attackers are bit-identical. Detection cost measured live: zero .

THE SINGLE MOST QUOTABLE LINE FROM THIS WHOLE HISTORY

Every live-run defect in this project was the controller's own bookkeeping feeding a working detector bad input. Not once were the detectors at fault. That distinction — “correctly flagged” versus “unable to be un-flagged”, “correctly isolated attacker” versus “wrongly starved honest node” — is the discipline that got the project from a total collapse to a perfect honest side in ten runs.

15 Limitations, stated before anyone asks

Volunteering these is worth more than the marks they cost. One overclaim discovered by an examiner taints every honest claim around it.

LIMITATION	DETAIL AND HONEST FRAMING
RAFT is built but not wired in	Core, TCP transport, 3-replica live cluster and safety proofs all exist and are demonstrated. The controller still commits single-replica — every block carries <code>raft_term = 0</code> . Three named runtime blockers; the swap is deliberately deferred. §8.3.
DDoS response is detection-only	Flooding is detected and correctly classified but not throttled. It needs a per-client-IP OpenFlow path; today's rate-limit machinery is per-edge-node. Deferred as real scope rather than half-built.
Detection latency is an upper bound	Onset is measured from agent launch, but the recording's t0 is controller start, which necessarily precedes it. The reported figure overstates the detector. Overstating is the safe direction; quietly subtracting an estimate would not be.
Scalability beyond 8 nodes is simulation	The box has 4 cores. N=64 live would measure the laptop, not the architecture. The sweep drives the real selector, but it is not a live run. What it supports is that the <i>selection rule</i> , not the hardware, is what stops scaling under argmax — not a capacity figure for 40 real machines.
Single run per configuration	Every live figure is one run at one seed. No confidence intervals. The paired-comparison machinery exists and produces the §7.4 statistics, but on the <i>simulation</i> harness; pointing it at live runs additionally needs a library version fix on this box.
Localhost, not multi-host	Three OS processes and real loopback TCP, not three separate machines. The code, the sockets and the process boundaries are real; only hardware separation is not.
The controller is a trusted single point	This is in genuine tension with “never trust”, and it is the standard SDN trust model. RAFT replication of ledger state is a <i>step</i> toward controller redundancy; full controller distribution is future work. Say this out loud — in a Zero-Trust-titled project, somebody will notice.
The adversary model has edges	We do not handle: collusion between edge nodes, adaptive adversaries that hold every signal just under threshold, on-off attacks tuned precisely against the EMA decay window, or attacks on the controller and dashboard themselves.
One false-positive flood tell	One honest device tripped the flood detector once, in the worst run, plausibly under retry pressure. Four later runs produced none — which supports that reading without proving it.
One detector threshold is not fully validated	A weighing threshold sits mid-band in a swept safe range, but the two most recent runs happened to be indifferent to it. Our own rule applies: indifferent means untested, not safe . We do not call it validated.

16 Anticipated panel questions and answers

Rehearse these. The first eight are near-certain.

Q Your title says Zero Trust, but your controller is fully trusted. Isn't that a contradiction?

A Yes, and we state it as a limitation rather than hiding it. It is the standard SDN trust model: the control plane is inside the trusted computing base. Our Zero Trust claim is scoped to the **serving infrastructure** — the edge nodes are never trusted, are verified continuously, and are verified on observed rather than claimed behaviour. Replicating ledger state with RAFT is a first step toward controller redundancy; full controller distribution is named future work.

Q RAFT is crash-fault tolerant, not Byzantine. A malicious replica can lie and RAFT will happily replicate it. So what does your blockchain actually secure?

A Two things, precisely: **tamper-evidence** — the hash chain and Merkle roots make retroactive falsification detectable and localisable to the seam — and **availability** under replica crashes. Both hold within a controller cluster that is itself in the trusted computing base. It is not permissionless, it does not tolerate Byzantine replicas, and it does not need to, because our threat model puts malicious behaviour at the edge nodes. If you follow up with “then why not a signed append-only log?” — a hash-chained Merkle log replicated by RAFT essentially *is* one, structured for per-record proofs and batch commits.

Q PRESENT-80 is an 80-bit cipher. That is below modern security margins.

A Correct, and we frame it accordingly: an ISO/IEC 29192-2 lightweight block cipher appropriate to severely constrained IoT devices, used here to demonstrate the challenge-response **architecture**. A production deployment would drop a 128-bit primitive behind the same Authenticator seam — and that seam already exists, which is why this is a swap and not a rewrite. Separately, the more interesting weakness is not the key length at all: the key is **fleet-wide**, so it authenticates possession of the key rather than the device. That is the gap source-IP pinning closes.

Q Two of your eight attacks were misclassified. Isn't that a failure?

A Both misses are in the **right family**, so the response would have been identical either way — the attacker was still detected, still quarantined, still contained in single-digit seconds. And the cause is a limit we characterised in advance: **containment quality and magnitude evidence are in direct tension**. The faster you isolate an attacker, the less evidence exists to say *how much* it was dropping. In that run our defence was faster and cleaner than ever — which is precisely what starved the classifier. We traded two label-precision points for a perfect honest side: zero honest mislabels, zero honest quarantines, zero route denials.

Q Two servers got no traffic at all. Isn't that a load-balancing flaw?

A There are two distinct zero-traffic cases and they must not be conflated. A **malicious** node receiving zero traffic is the isolation working correctly. A **healthy** node receiving zero traffic is a genuine flaw — starvation — and we found it, characterised its root cause (argmax plus trust-coupled selection produces rich-get-richer lock-in onto ~2 nodes regardless of fleet size), and fixed it with power-of-two-choices plus ϵ -exploration. Note also that starved healthy servers receive **zero packets** — they do not *drop* anything. Only a quarantined node gets drop rules.

Q Your system is slower than round-robin when there is no attacker. Doesn't that undermine the whole thing?

A It is, by 10.7% on SLO violations in the clean scenario, and we report it. The cause is not the trust term — it is staleness in the load signal that **every** load-reactive strategy reads, including least-connections, which does far worse. We isolated it with a poll-interval sweep: halving the poll interval closes most of the gap. The trade is explicit: in a clean network you pay a small cost; under any of the four attack scenarios you win by 15–69%.

Q Why doesn't your dashboard show every packet?

A Because it structurally cannot, and animating one would be a lie. Once a flow rule is installed, matching packets are forwarded entirely in the data plane and never reach the controller again. We therefore drive the animation from 1 Hz flow-statistics counter deltas and print the measured packets-per-second on screen so the claim is checkable. This is Finding 3 — a real observation about SDN observability.

Q How do you know your results aren't just the trust mechanism getting lucky on one run?

A Three independent lines of evidence. First, a **controlled comparison**: same topology, same devices, same attacks on the same schedule, same launching code, differing only in whether anything acts on trust. Second, a **600-run seeded statistical harness** driving the real selector and the real calculator — 14 of 16 comparisons favour the system at $p_{\text{adj}} = 7.45 \times 10^{-9}$ with the system winning 30 of 30 paired seeds. Third, an **ablation**: the same selector with only the trust term removed, which is what attributes the gain to trust rather than to load-aware routing generally. And we state the limitation plainly — every *live* figure is a single seed.

Q What happens if a quarantined node was innocent?

A It gets out. **Probation** is the half-open leg of a circuit breaker: a node isolated on the trust rail only, whose anomaly score has since fallen below the gate, receives one trial task per interval, and the trial flow installs *above* the quarantine drop rules so it is not black-holed by the rules it exists to escape. In our best run that produced 23 recoveries from 25 quarantines. We had to invent this because in an early run quarantine was an **absorbing state** — isolation starved the very evidence needed to leave it.

Q Why os-ken and not Ryu, as your deck specified?

A The target OS ships Python 3.14; Ryu cannot run above 3.13 and has no package for it. os-ken is the actively-maintained fork of Ryu with an identical API. It is an environment-driven engineering decision, documented, with no `ryu` dependency anywhere in the repository.

Q Would this work at 1 000 nodes?

A We do not know, and we will not claim it. What we measured: at 8 servers we measure live; beyond that we **simulate the fleet while executing the real selector**, up to $N=64$. That supports the claim that the **selection rule**, not the hardware, is what stops scaling under argmax — power-of-two-choices scales linearly where argmax saturates. It does **not** support a capacity figure for real machines. Running $N=64$ on a 4-core box would measure the laptop, not the architecture.

Q Where would this actually be deployed?

A MEC and the 5G telco edge first — the mapping is nearly one-to-one: our controller is the orchestrator, our edge servers are MEC hosts, our IoT clients are user equipment, and MEC orchestrators route on self-reported host load today. Industrial and municipal edge second. Explicitly **not** the public cloud, where the operator owns and attests its own hardware, and not consumer IoT, which would be buzzword marketing.

Q Isn't this just a health check with extra steps?

A A health check asks the backend a question and believes the answer, and its verdict is binary. Ours computes a graded score from evidence the backend cannot fabricate — completed versus failed outcomes, observed response time against the fleet median, and the gap between claimed load and a duty cycle the controller derives itself — and adds a second, independent gate that can act when the score arithmetically cannot. The case that separates them is our central one: a host returning perfect responses while lying about capacity passes every health check ever written.

Q Your attacker share is still 12.5%. Why not zero?

A Because attackers are behaviourally identical to honest nodes before onset — every attack in our configuration has a **delayed onset** precisely so that the detectors must notice a change rather than recognise a permanently odd node. The 12.5% is overwhelmingly pre-onset traffic plus the seconds before containment. The baseline's 30.5% is the same traffic plus everything that continued flowing for the rest of the run because nothing could stop it.

Q What is the single biggest weakness of this project?

A That every live figure comes from a single run at a single seed. We have statistical rigour on the simulation harness and fidelity on the live runs, but not both on the same measurement. That is the honest answer, and it is the first thing we would fix with more time — ahead of wiring in RAFT and ahead of DDoS throttling.

Q What did you personally learn from building this?

A That the hard part was never the detectors. Every single defect across eleven live runs was the controller's own bookkeeping feeding a working detector bad input — a stale counter, a rejuvenated timestamp, a task charged to the wrong node. The discipline that got us out was refusing to conflate two things that look alike in an average: “correctly isolated attacker” versus “wrongly starved honest node”, and “correctly flagged” versus “unable to be un-flagged”. Every metric in this project that touches quarantine reports those separately.

17 What to demonstrate live, and the fallbacks

17.1 The demonstration, in order

1. **Live Mininet run** — `sudo python3 run_demo.py --mode mininet --config config/params_trust_full.yaml`
2. **The dashboard** at `http://localhost:8081/` — quarantine, re-steer and block commits are visible as they happen. Point at the trust curves diverging when each attack arms.
3. **Real flow tables** — `mininet> dpctl dump-flows -O OpenFlow13 | grep cookie=0x5a`. This is the proof that the decision is enforced in the data plane and not simulated.
4. **The API by cURL** — `iot1 curl -s http://10.0.99.254:8081/node/status`
5. **The ledger tamper control** — break the chain in the browser and watch the break localise to the seam. The page recomputes SHA-256 itself rather than trusting the controller's own `valid` flag.
6. **Recorded RAFT failover** — `python3 -m blockchain.raft_timeline --duration-s 45 --kill-at-s 15 --restart-at-s 30`
7. **The test suite** — `python3 -m pytest -q`, 50.7 s, 993 passing.

17.2 Environment prerequisites — check every one before the room

1. `sudo service openvswitch-switch start` — WSL does not auto-start it, and without it the topology hangs with no visible error.
2. `sudo modprobe -a sch_htb sch_netem sch_tbf sch_prio ifb` — note the `-a`; without it only the first module loads and the rest are treated as parameters.
3. `sudo mn -c` between runs, to clear stale veth state.
4. **Controller first, topology second.** A switch that connects to nothing installs no rules, and every connectivity test silently times out — which looks exactly like a freeze.
5. Live mode runs under `sudo`, so outputs land owned by root. The next non-sudo controller then dies with a permission error that reads like a broken controller. The runner chowns artefacts back at the end of a run; older ones need `sudo chown -R $USER data logs`.

FALLBACKS, IN ORDER OF PREFERENCE

1. Replay a recorded run into the live dashboard — `python3 -m dashboard.replay data/events.jsonl --loop`, then open `localhost:8082`. It exercises every panel with real recorded events and needs no `sudo` and no Mininet.
2. Run the four offline scorers against an archived recording — NFR report, confusion matrix, availability report, interval report. None needs the controller stack installed.
3. Run the 600-run statistical harness — it takes about 40 seconds and needs nothing but Python.
4. The figures in this document and the generated slide figures under `ppt/figures/`.

Archive the recording before any new run. Live mode unlinks `data/events.jsonl` at startup and `data/` is gitignored — copy it to `data/events_runN.jsonl` the moment a run finishes, or its numbers cannot be re-derived.

18 Glossary and provenance

18.1 Terms a panellist may ask you to define

SDN — Software-Defined Networking: the forwarding decisions are made by a central controller, and switches only execute installed rules.

OpenFlow 1.3 — the protocol between controller and switch. Rules match on packet fields and carry actions, priorities and cookies.

Flow rule / cookie — an installed match-action entry. A cookie is an identifier we attach so all of a node's rules can be deleted in one operation.

VIP — Virtual IP. Clients connect to one address; the controller rewrites the destination, so the client never learns and never chooses which server serves it.

PacketIn — what a switch sends the controller when no rule matches. This is where a routing decision is made.

EMA ($\lambda = 0.85$) — exponential moving average; recent evidence dominates, older evidence decays smoothly.

Jain fairness index — 1.0 means perfectly equal shares; lower means more skew. Must be read over a stated population (§7.3).

Merkle root — one hash summarising a set of records, allowing any single record to be proven a member without revealing the rest.

Tamper-evident — modification is *detectable*, not prevented. This is the correct word for our ledger.

Crash-fault vs Byzantine — crash-fault tolerance survives replicas that stop; Byzantine tolerance survives replicas that *lie*. Ours is crash-fault.

PRESENT-80 — an ISO/IEC 29192-2 lightweight block cipher for constrained devices, 80-bit key.

TOFU — trust on first use: the first host to claim an identity owns it. Our fallback where no provisioned roster entry exists.

Power-of-two-choices — sample two candidates at random, take the better. Provably far better tail behaviour than picking the single best from a stale view.

Wilcoxon signed-rank / Holm-Bonferroni — a paired non-parametric test, and the correction applied because testing against four baselines four times inflates the false-positive rate from 5% to ~19%.

18.2 Where every number in this document comes from

CLAIM	SOURCE
All §7 comparison figures, containment times, per-server trust, spoof outcome	<code>data/comparison/comparison.txt</code> — the 2026-09-05 pair: baseline 307.5 s (<code>base_events.jsonl</code>) vs zero-trust 313.2 s (<code>events.jsonl</code>)
All four NFR measurements in §8.1	<code>data/nfr_report.txt</code> , generated by <code>evaluation/nfr_report.py</code> from the same recording
RAFT failover, commit counts, per-commit cost	<code>data/raft_timeline.jsonl</code> , 45 s run of 2026-09-07, plotted by <code>evaluation/plot_raft_timeline.py</code>
The 600-run statistics, SLO reductions, p-values, poll sweep	<code>docs/EVALUATION.md</code> , produced by <code>evaluation/baseline.py</code> + <code>evaluation/stats.py</code>
Scalability figures to N=64	<code>evaluation/scalability_sweep.py</code> — a queueing simulation driving the real selector , not a live run
Test counts	<code>pytest -q</code> : 1 004 collected, 993 passed, 11 skipped, 0 failed
The eleven-run history in §14	<code>SOURCE_OF_TRUTH.md</code> §5, <code>docs/LIVE_RUN_8_40_3.md</code> , and the archived per-run recordings
All twelve figures	Generated by <code>ppt/fig_architecture.py</code> , <code>ppt/fig_flows.py</code> , <code>ppt/fig_results.py</code> — never hand-drawn

THE THREE SENTENCES TO CLOSE ON, IF YOU ARE ASKED TO SUMMARISE

1. Orchestrators route work using what nodes say about themselves; we built a control plane that routes on what it observes, and enforces that verdict in the data plane in under a second.
2. We proved our own published trust formula insufficient — a competent liar floors at 0.44 against a 0.30 threshold — and kept the finding rather than retuning the weights until the demo worked.
3. We measured it against a full control arm running the same code with enforcement removed: honest-device availability 85.7% → 99.0%, starved devices 5 → 0, every attacker contained in 8-11 seconds where the baseline contained none of them, ever.

Zero Trust-Enabled SDN Architecture for Secure and Intelligent Load Balancing in Edge Networks · 23CSE498 Project Phase 2, Panel Review 1 · Team 50 · briefing compiled 2026-09-08. Figures from `ppt/figures/` . Numbers as cited in §18.2; where a figure is a simulation, an estimate or an upper bound, it is labelled as such at the point of use.